

Metaprogramming in Java: Introspection and Annotations

Lecture 01 ; Advanced Programming

Ayoub FRIHAOUI
ayoub.frihaoui@lecnam.net

Adapted from the original French lecture by
Rezak AZIZ rezak.aziz@lecnam.net
Conservatoire national des arts et métiers (CNAM Paris)

Original material © Rezak AZIZ, CNAM Paris.
Source: <https://rezakaziz.github.io/PROG-AVANC/>

Course objectives

- Understand **metaprogramming**: writing code that manipulates code;
- Discover two main mechanisms in Java:
 - **Introspection** (the Reflection API);
 - **Annotations** and how they are processed;
- Identify their uses in modern frameworks: Spring, Hibernate, JUnit, Lombok.

What is metaprogramming?

Definition

The ability of a program to **inspect, modify or generate code**.

In Java, metaprogramming is done through:

- ➊ **Introspection** (inspecting code at runtime);
- ➋ **Annotations** (adding metadata);
- ➌ **Dynamic proxies** (generating classes at runtime).

Used heavily in:

- **Spring** (dependency injection);
- **Hibernate** (object–relational mapping);
- **JUnit** (automatic discovery of tests).

Why should you care?

You will rarely write raw reflection. You will use it every day.

Frameworks look like “magic”: `@Test` methods run themselves, `@Entity` classes become SQL tables, `@Autowired` fields are filled in. That magic *is* metaprogramming.

What this will serve you in practice

- **Read framework code** instead of fearing it (Spring, JPA, JUnit, Lombok).
- **Debug** mapping errors, missing annotations, and “why is this not injected?”.
- **Interviews**: “How does JUnit find your tests?” ; you will have a precise answer.
- **Later in this module**: dependency injection (Lecture 08) is built on the same idea.

Rule of the lab

Do *not* sprinkle introspection into every class. Prefer readable code. Frameworks already did the hard part ; this lecture shows *how*.

Introspection

Principle of introspection

Simple definition

Introspection lets a Java program **explore its own structure**:

- know the name of a class;
- see the methods and fields it contains;
- and even execute or modify them.

This is done through the class `java.lang.Class`
(package `java.lang.reflect`: `Method`, `Field`, `Constructor`).

Example: observing a class

```
// Retrieve the description of the String class
Class<String> clazz = String.class;

// Print the class name
System.out.println("Class name: " + clazz.getName());

// List all public methods
for (Method m : clazz.getMethods()) {
    System.out.println("Method: " + m.getName());
}
```

Explanation

This program “observes” the class `String`:

- it discovers its name (`java.lang.String`);
- it prints the list of all its **public** methods.

The main methods of class `Class`

- `getMethod(String name, Class... parameterTypes)`
Retrieves a **public** method by name and parameter types.
Example: `clazz.getMethod("substring", int.class, int.class);`
- `getMethods()`
Retrieves all **public** methods of the class (including inherited ones).
- `getDeclaredAnnotations()`
Retrieves all annotations **declared** on the class.

Note: objects of type `Method` also have useful methods such as `getName()`, `getReturnType()` and `invoke(...)`.

Dynamic method invocation

```
// A simple class
class Person {
    private String name = "Alice";
    public void sayHello() {
        System.out.println("Hello " + name);
    }
}

Class<?> clazz = Class.forName("Person");           // load the class
Object p = clazz.getDeclaredConstructor().newInstance(); // create a Person
Method m = clazz.getMethod("sayHello");             // public method
m.invoke(p);                                         // Hello Alice
```

Modifying a private field

```
Field f = clazz.getDeclaredField("name");  
f.setAccessible(true);    // allow access to the private field  
f.set(p, "Bob");          // change the field value  
m.invoke(p);              // Hello Bob
```

To remember

- `getMethod(...)`: retrieve a public method by name;
- `invoke(obj)`: execute the method on a given object;
- `getDeclaredField(...)`: access a field even if it is private.

Note: these mechanisms are used inside frameworks such as Spring or Hibernate.

Annotations

Annotations in Java

Role

Annotations add **metadata** to the code: they inform the compiler, the IDE, or the program itself.

Examples: `@Override`, `@Deprecated`, `@Test`, `@WebServlet`.

They can be used:

- **at compile time** (checks, code generation). e.g. `@Override`;
- **at runtime** (via introspection). e.g. `@WebServlet`.

Annotation arguments

General principle

Annotations may take arguments, which are Java objects.

Special cases

If the annotation defines a single argument named `value`, the parameter name may be omitted.

An array of values is written between braces: `{...}`.

Examples of annotations with arguments

// 1. several named arguments

```
@Test(expected =  
    VerificationException.class,  
    timeout = 1000)
```

// 2. a single "value" argument

```
@SuppressWarnings("unchecked")
```

// 3. an array of values

```
@SuppressWarnings(value = {  
    "unchecked", "deprecation"})
```

Defining a custom annotation

Declaration of the annotation

```
// Visible at runtime
@Retention(RetentionPolicy.RUNTIME)
// Only usable on methods
@Target(ElementType.METHOD)
public @interface Loggable {
    // Optional argument "value"
    String value()
        default "Method call";
}
```

Example of use

```
public class Service {
    @Loggable("Main processing")
    public void process() {
        System.out.println(
            "Processing...");
    }
}
```

The annotation `@Loggable` marks methods to be tracked. They can later be detected by introspection, for example to write a message into an execution log.

Retention of annotations

Three possibilities:

- SOURCE — no trace of the annotation remains in the Java bytecode. Possible example: `@SuppressWarnings`.
- CLASS — the annotation is kept in the bytecode but is **not** readable at runtime.
- RUNTIME — the annotation is kept at runtime; it is visible to the program (e.g. `@Test`, `@WebServlet`).

Reading an annotation at runtime

```
Service s = new Service();

// Walk the methods of the class
for (Method m : s.getClass().getDeclaredMethods()) {
    // Check whether the method carries @Loggable
    if (m.isAnnotationPresent(Loggable.class)) {
        Loggable ann = m.getAnnotation(Loggable.class);
        System.out.println("[LOG] " + ann.value());
        m.invoke(s);    // execute the method
    }
}
```

Observation

In the previous example:

- annotations were read **through introspection**;
- Class and Method provide:
 - `getAnnotation(A)` ; takes the annotation class, returns an annotation object or `null` if it is absent;
 - `getAnnotations()` ; returns an array of all annotations on the class or the method;
- the fields of the annotation are then accessible (e.g. `ann.value()`).

Annotations and inheritance

- Annotations on **methods** are *never* inherited;
- Annotations on **classes** *may* be inherited (if the annotation itself is meta-annotated with `@Inherited`).

Consequence

`getAnnotation` on a subclass sees a class-level `@Inherited` annotation declared on the superclass.

It does *not* see a method annotation declared only on the superclass method.

Concrete example: Hibernate

Hibernate: what is it?

Definition

Hibernate is a Java framework that simplifies access to databases. It is part of the **JPA** (Java Persistence API) specification.

- It lets you persist Java objects in a relational database **without writing SQL by hand**.
- It performs the automatic mapping between:
 - Java **classes** and SQL **tables**;
 - Java **fields** and SQL **columns**.

Hibernate and annotations

Hibernate uses metaprogramming to **read annotations** and generate the corresponding SQL. The annotations define how the Java class is stored in the database.

```
@Entity
@Table(name = "clients")
public class Client {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "name")
    private String name;
}
```

→ Hibernate reads these annotations by introspection and builds the SQL queries to insert, update or read the objects.

How Hibernate uses metaprogramming

- 1 Analyses the annotated classes (`@Entity`, `@Column`, `@Id`).
- 2 Dynamically creates the matching SQL.
- 3 Calls constructors and setters to rebuild objects from the SQL result.

Example of automatic mapping

Java class	SQL table
<code>Client.name</code>	<code>name VARCHAR(255)</code>
<code>Client.id</code>	<code>id INT AUTO_INCREMENT</code>